

Blokus Trigon Online

From PRD to deployed, polished web app in 60 hours

A full-stack multiplayer board game built from **0 to fully polished in < 2 months** (to playable local game in < 1 week) — using React, Node.js, real-time multiplayer, AI opponents, and full CI/CD.

MARCH 22 – MAY 20, 2026

SOLO DEVELOPED BY SANDRA SHAW

[View on GitHub](#)

[Play Blokus Trigon Online](#)

[Read the PRD](#)

Development lifecycle at a glance

4

Phases

Core engine local game → Multiplayer → AI → Polish

~60

Hours Logged

Across 8 weeks of evening & weekend sessions

60+

Git Commits

Version-controlled from V9 through final release

2

Deploy Platforms

Railway (backend) + Vercel (frontend)

Blokus was developed through four phases

1

Phase 1

Core game engine for local game, piece placement, rotation

2

Phase 2

Turn system, placement rules enforcement, game state

3

Phase 3

Online multiplayer, rooms, real-time sync, deployment

4

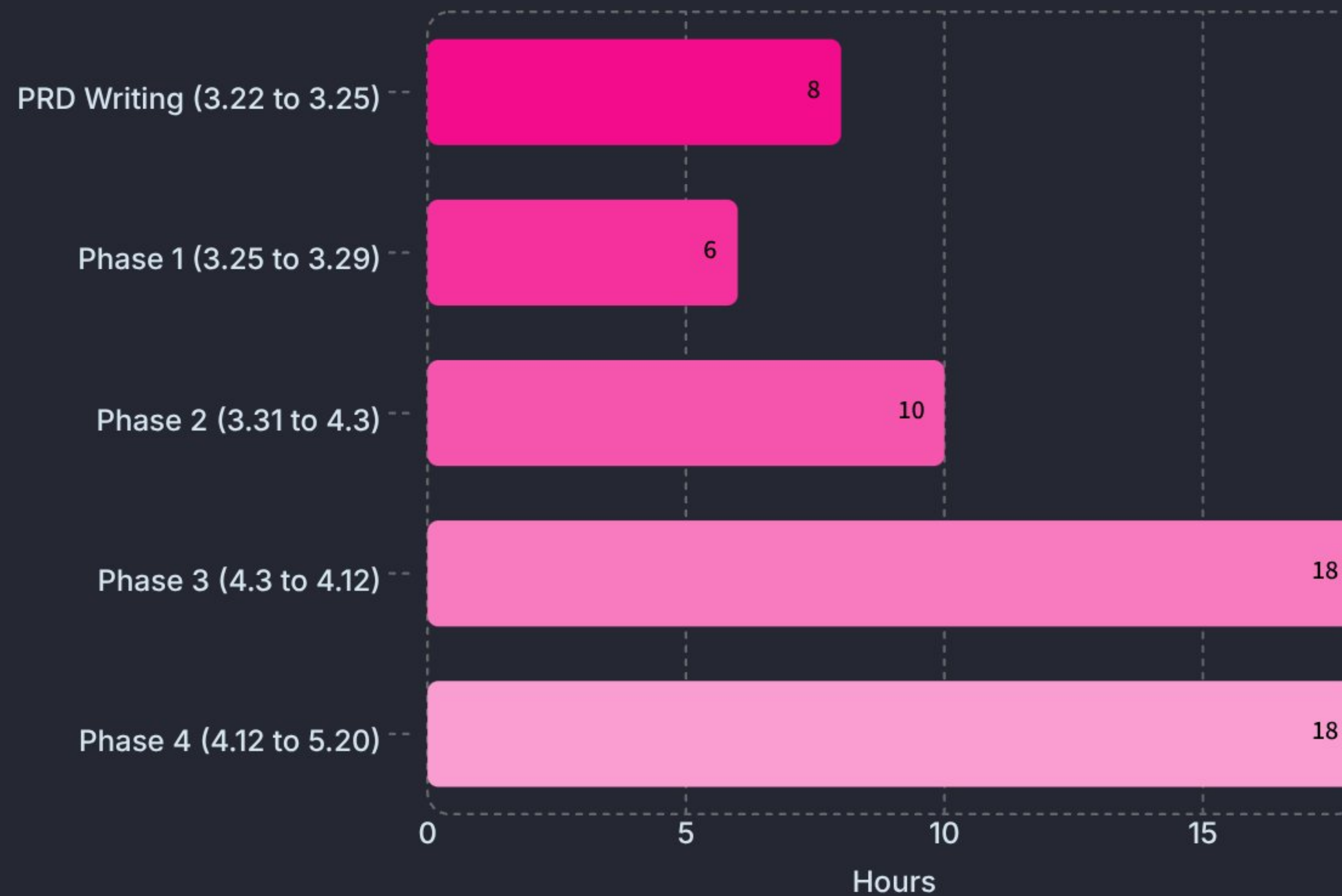
Phase 4

AI opponents, UI polish, timers, how-to-play modal

Each phase was PRD-driven — architecture was designed from day one to support the final state: hosted, online, with AI players.

It took an average of 13 hours per phase

60 total hours spent



Where the time went

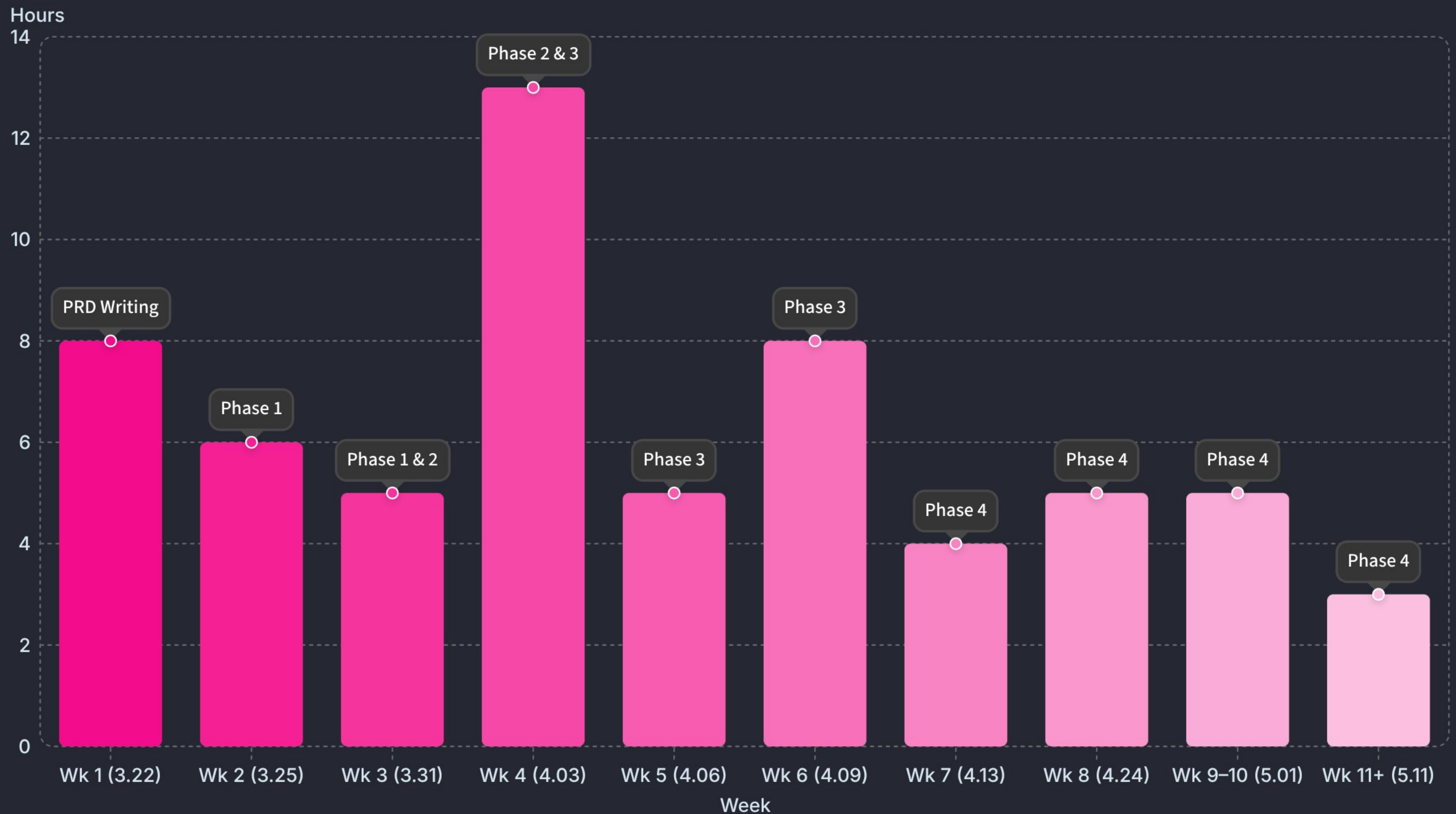
PRD Writing: initial phase, set the scope, architecture, and final target state before a single line of code.

Phase 1: focused on the core game engine and piece rotation logic in local game. Rules enforced in **Phase 2**.

Phase 3: most complex, requiring real-time socket architecture and cross-platform deployment.

Phase 4: stretched longest in calendar time, with sessions spread across evenings as nice-to-have enhancements and polish accumulated.

Time spent tapered off as the game stabilized



3.25.26 (Wk 2)

1st test run on local dev

3.31.26 (Wk 3)

1st full local game

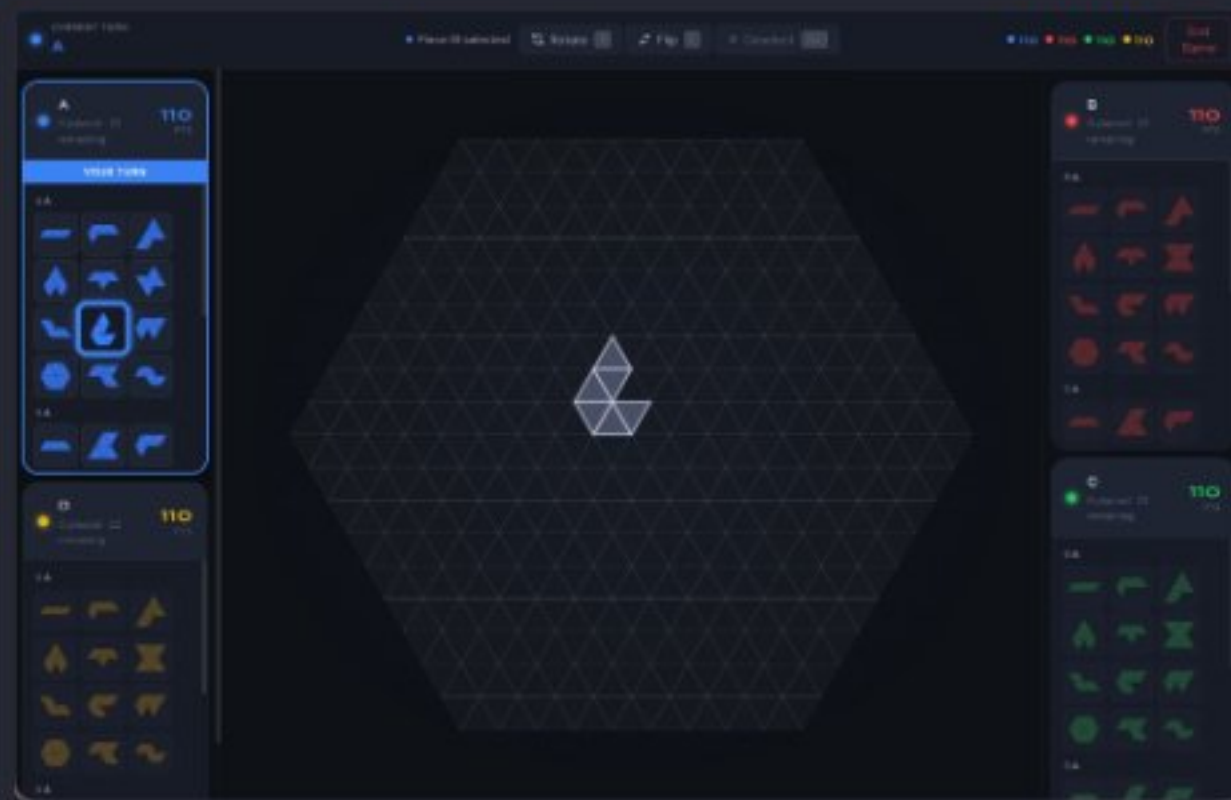
4.6.26 (Wk 5)

1st full multiplayer online game

Work was concentrated in **late-night** and **weekend sessions**. **Week 4** (deployment + Phase 3) was the single most active week.

The first version was very basic

Visual progression from raw engine to polished multiplayer product:



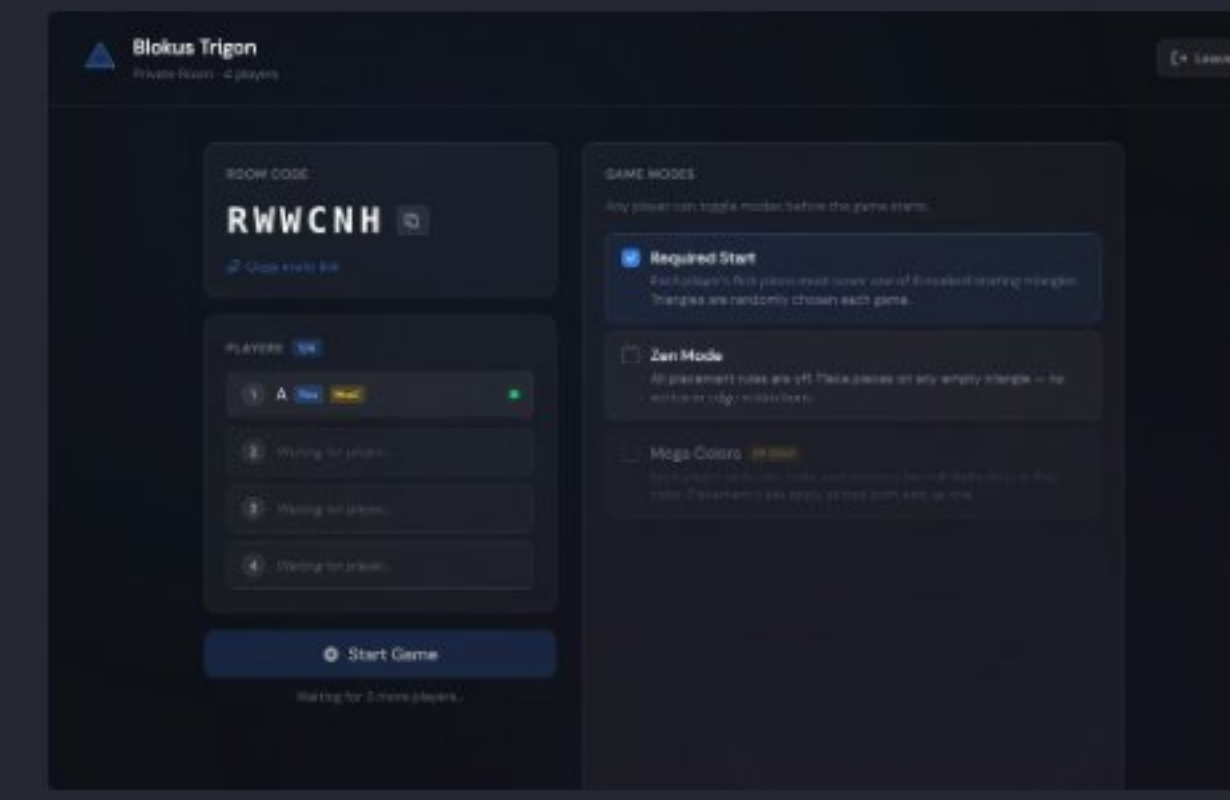
Phase 1: Core Engine

Piece placement & rotation
on triangular grid



Phase 2: Local Game

Turn system, game state,
rule enforcement



Phase 3: Online Play

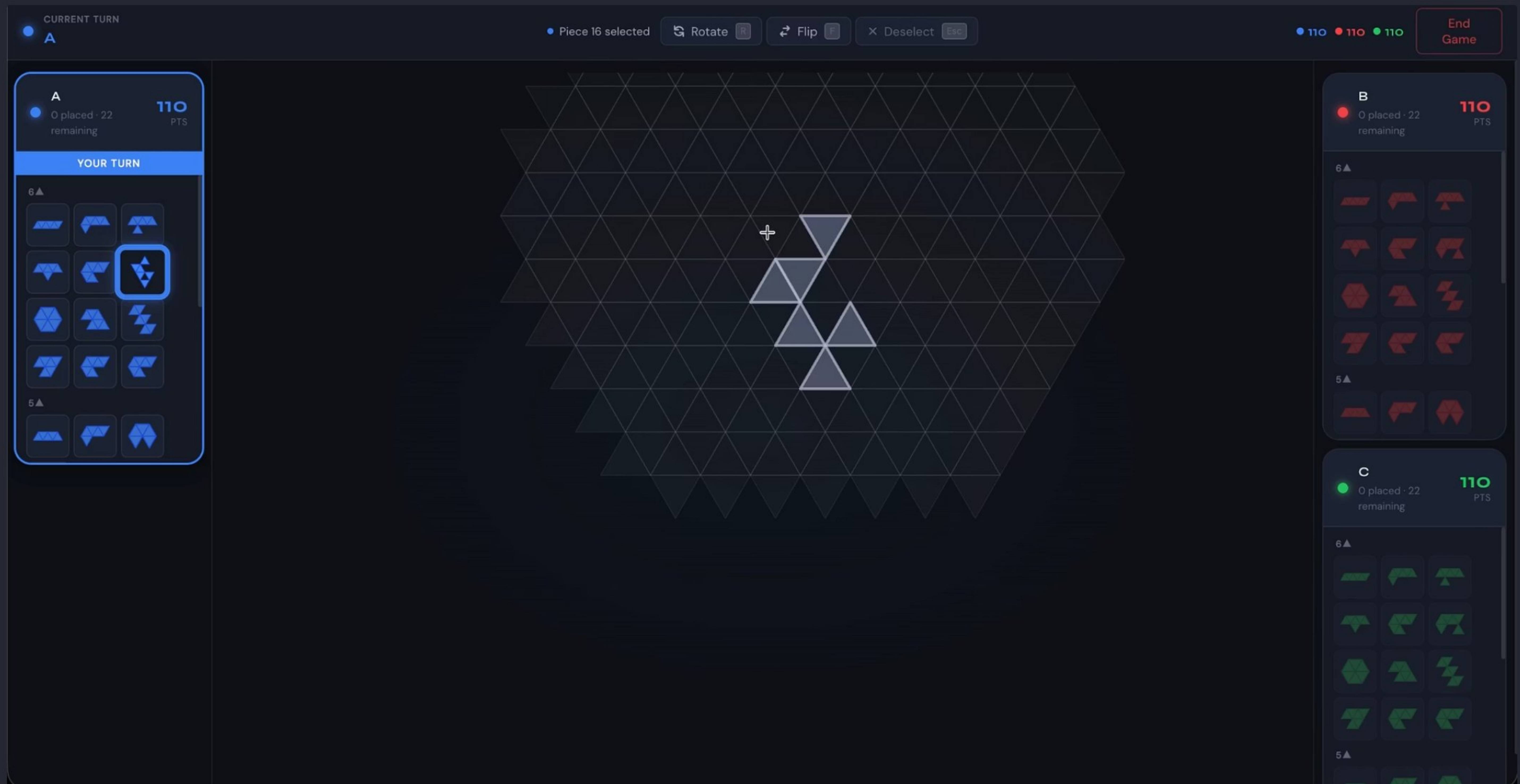
Room creation, real-time
sync, deployed to web



Phase 4: AI + Polish

AI opponents, timers, how-
to-play, final UI

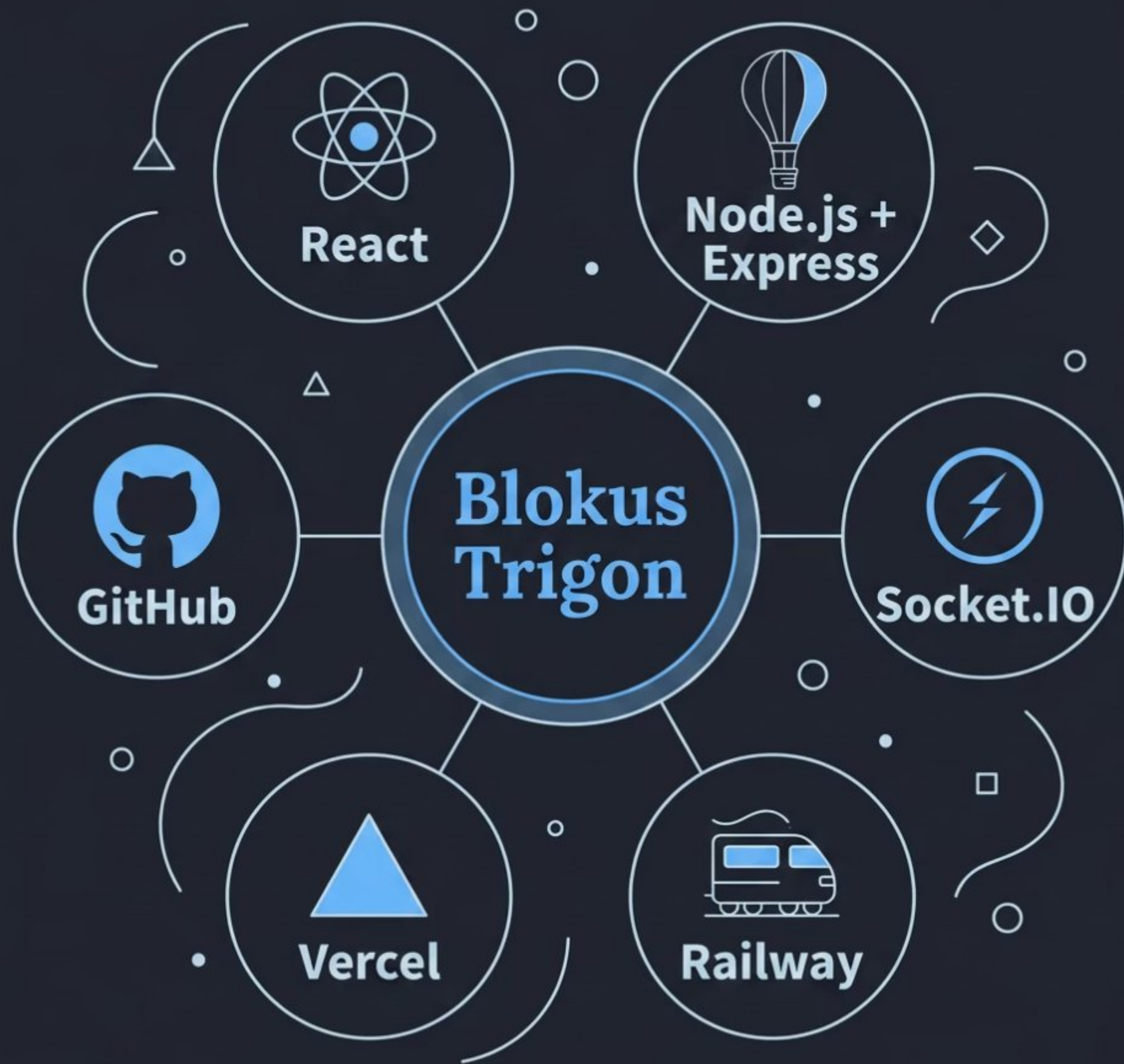
And the very first deployment...not exactly there



What is the board doing with the spiky edges? Why is half of it cut off? Why are some pieces completely untethered?? And don't even get me started about how the shape changes when you try rotating or flipping.

But it always starts somewhere...right?

A modern tech stack with CI/CD was leveraged



Architecture-first thinking

The stack was chosen on day one to support the **end state**, not just the MVP.

Socket.io enabled real-time multiplayer.

Railway + **Vercel** gave a clean split between backend and frontend hosting.

Git**Hub** for worktrees, version control, commit history, and deployments.

- ① AI-assisted: **Claude Code** was used as a code + design collaborator for UI modals and architecture decisions throughout development.



Some hurdles took a lot to debug

Vertex-to-Corner Bug

The hardest bug in the project — a triangular midpoint edge case that took **3 separate sessions** across 4 days to finally resolve, ultimately using Grok to break through.

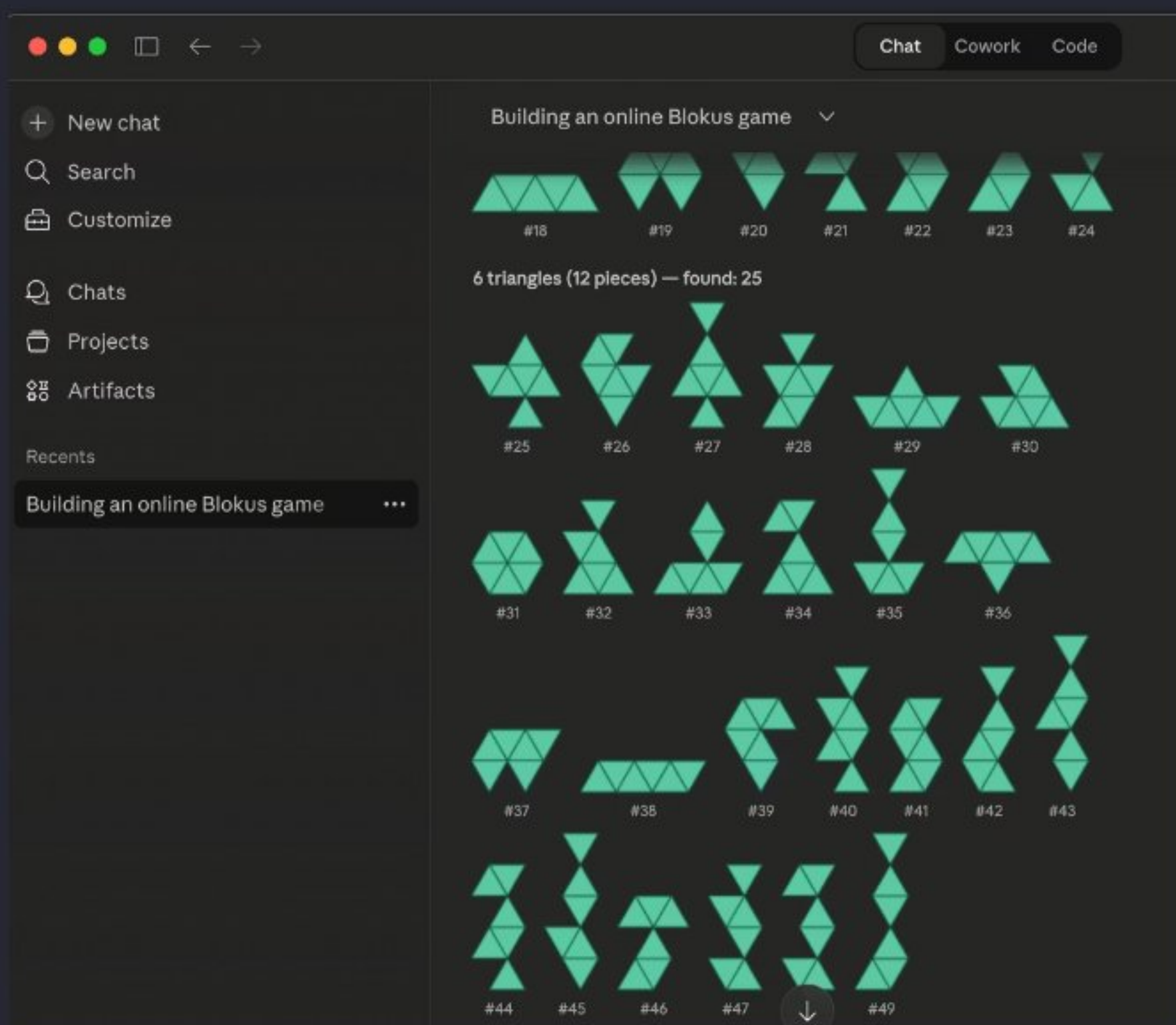
Defining the 22 Pieces

This was harder than expected — getting the 22 pieces with the right geometry and retaining its shape whenever it floats across the board, is flipped, or is rotated.

Don't ask why there were 49 "unique" pieces generated...

Real-Time Sync Complexity

Room creation, color selection, player order, and rejoin logic all required careful socket event sequencing to avoid race conditions.



Blokus took PdM vibe coding to a new level



Design for the End State

Choosing a stack that supports AI and online hosting from day one prevented costly rewrites later. Asking those questions to AI assisted coding environments was extremely helpful and enlightening.



AI as a Collaborator

Claude and Grok astronomically accelerated design and debugging — but context limits and hallucinations required human judgment to navigate.



PRD-Driven Development

Writing a full PRD before coding kept phases focused and gave a clear definition of "done" for each milestone. Updating the PRD with newer information and decisions was also important for future context.



Document Everything

Maintaining a bug log, change log, and running list of enhancements made it easy to track and realize progress across sessions.

Tldr; I went from idea to fully polished live — in under 60 hours

Play It

Deployed on Railway + Vercel: fully playable online with friends or AI opponents.

Open Source

Full commit history, PRD, and docs available on GitHub at **[sandstormbit/blokus-trigon-online](#)**.

What's Next

Smarter AI, newer game modes, and continued polish based on play-testing feedback.

npm run dev!



Blokus Trigon

STRATEGY · TERRITORY · TRIANGLES

[+ Create Game](#)

[→ Join Game](#)

[→ Local Game](#)

[① How to Play](#)

For educational purposes only. Online adaptation of Blokus Trigon by Mattel.

Appendix

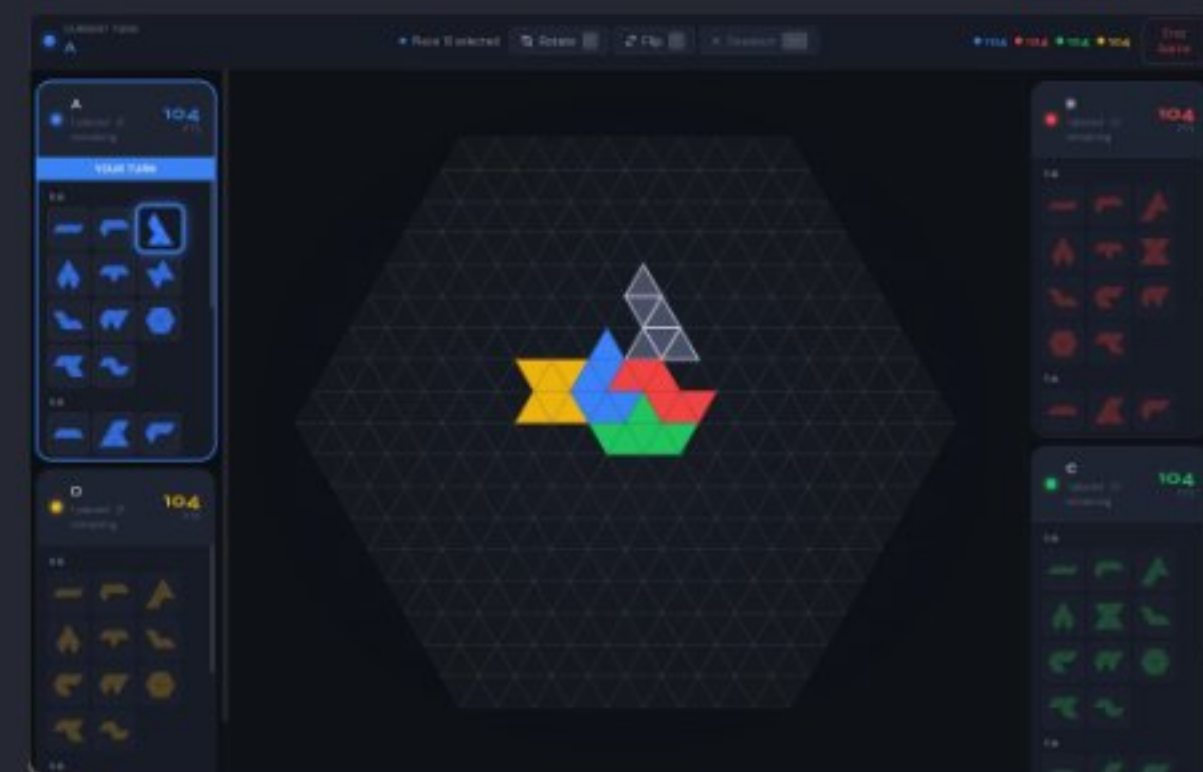
Phase 1: Core Engine

Piece placement & rotation on triangular grid

Game Setup

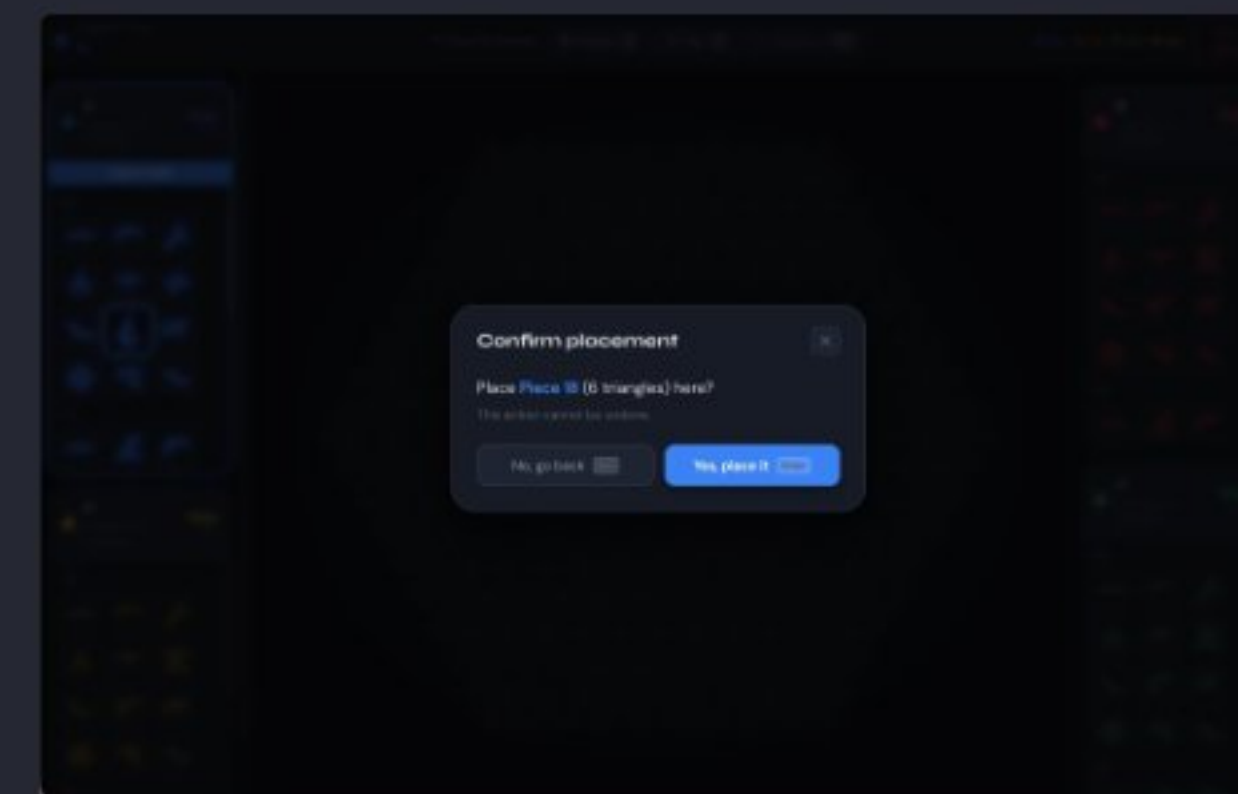


Piece Placement



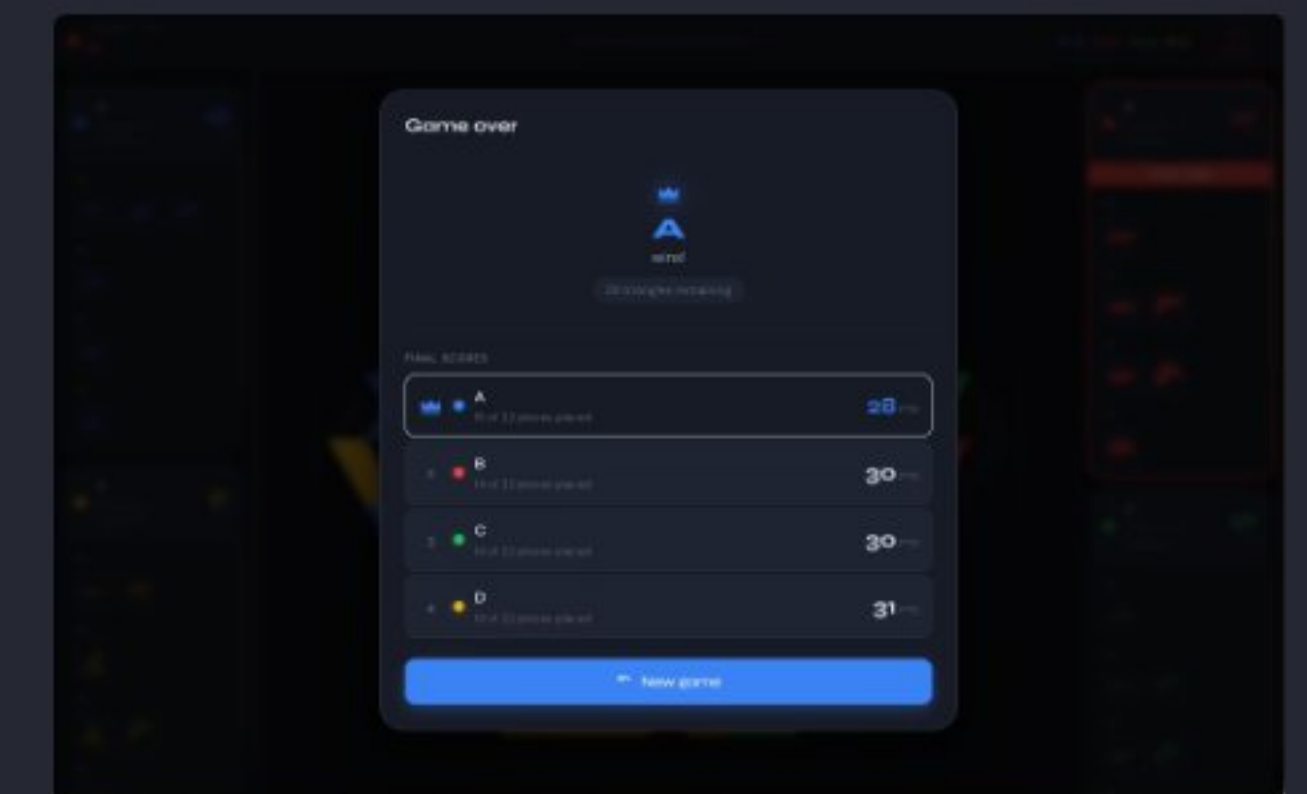
No rule enforcement, place anywhere on board

Confirmation



No undos, one click to move to next player

End Game Scoring



No automatic end game detection yet

Phase 2: Local Game

Turn system, game state, rule enforcement

Game Setup



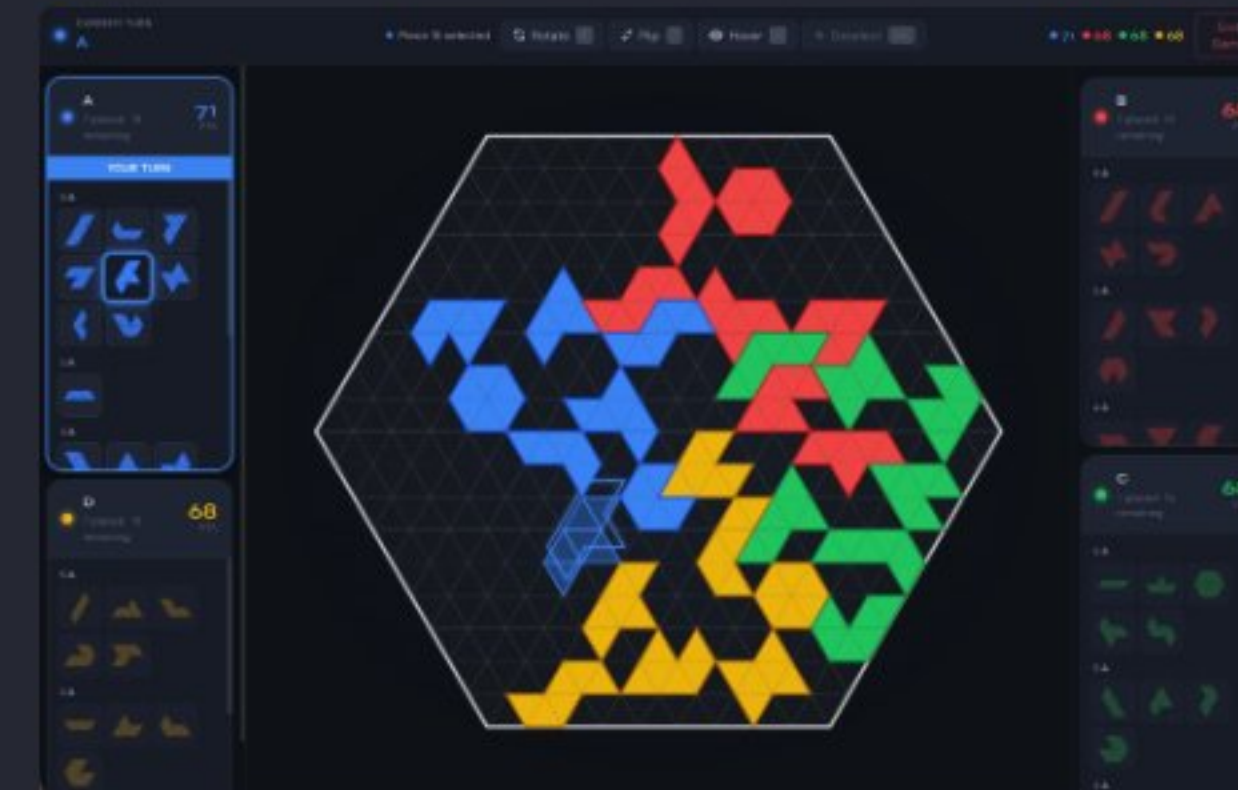
2, 3, or 4 player game, added
3 game modes

Piece Movement



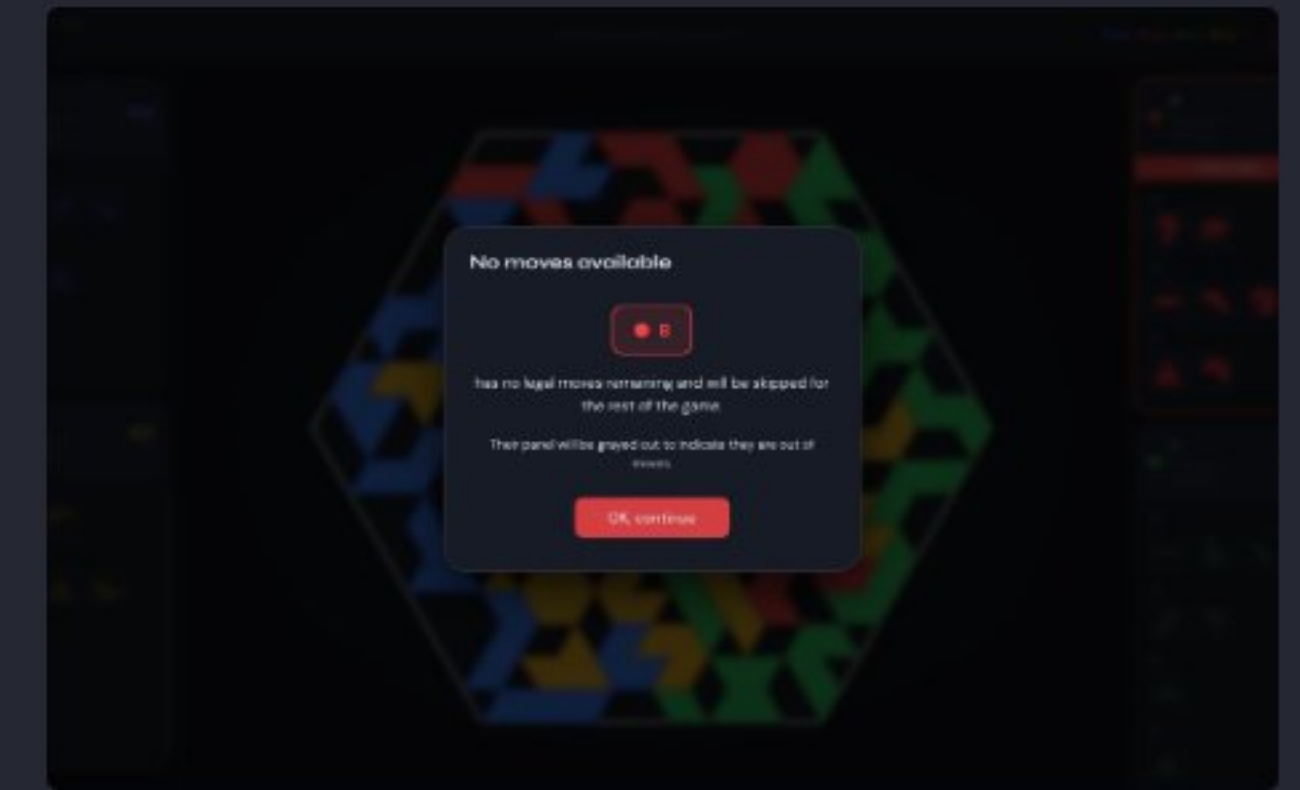
Added hover preview to see
shape at all times

Placement Validation



Highlighted to show a legal
move, gray to show it is not

No More Moves Skip

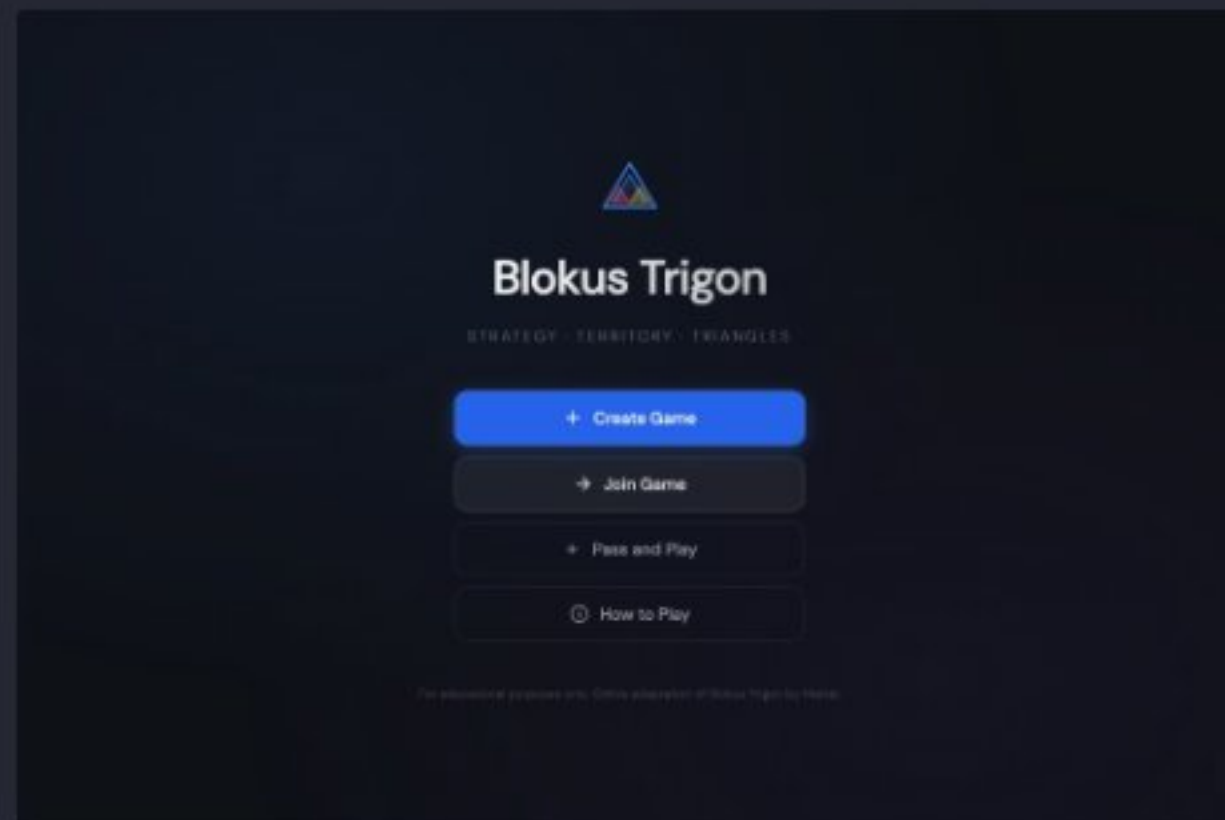


Skip turn if player has no
more legal moves

Phase 3: Online Play

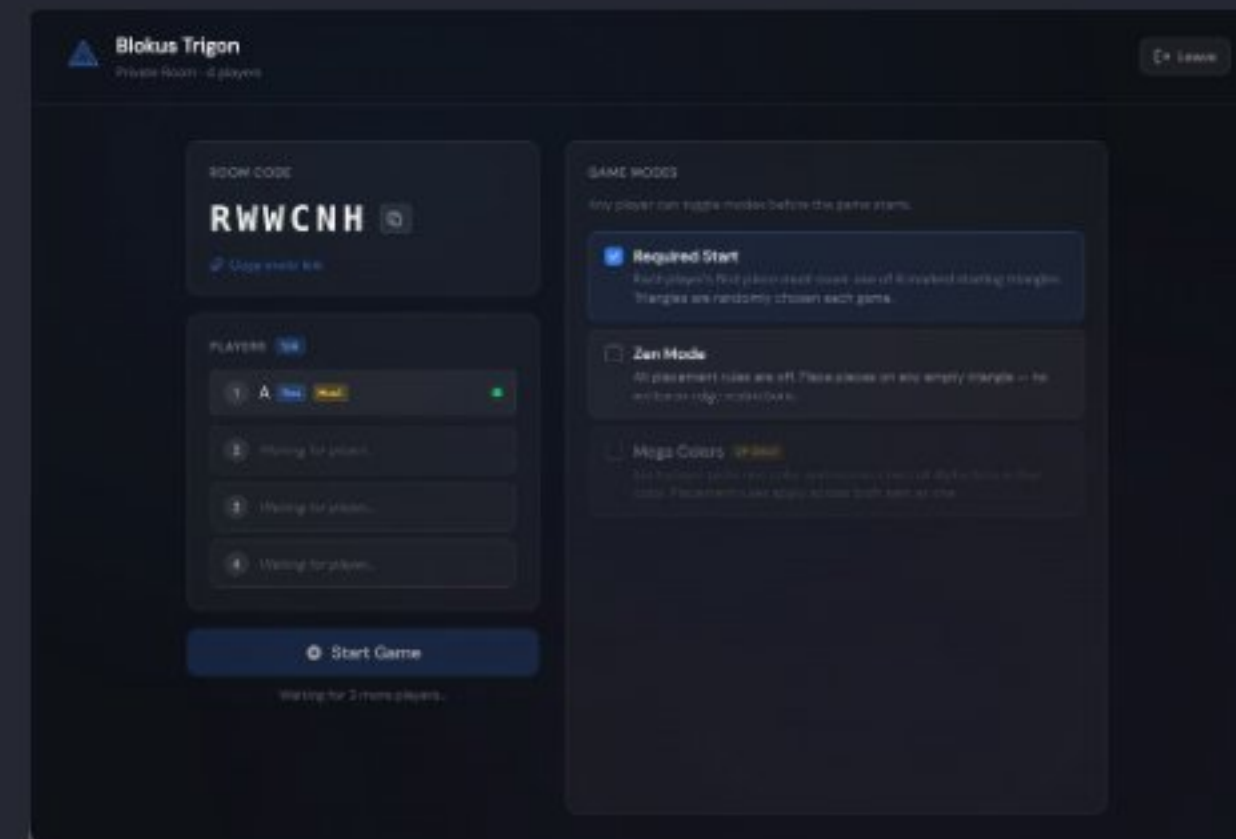
Room creation, real-time sync, deployed to web

Game Setup



Create game, join game,
local game, or how to play

Game Lobby Room



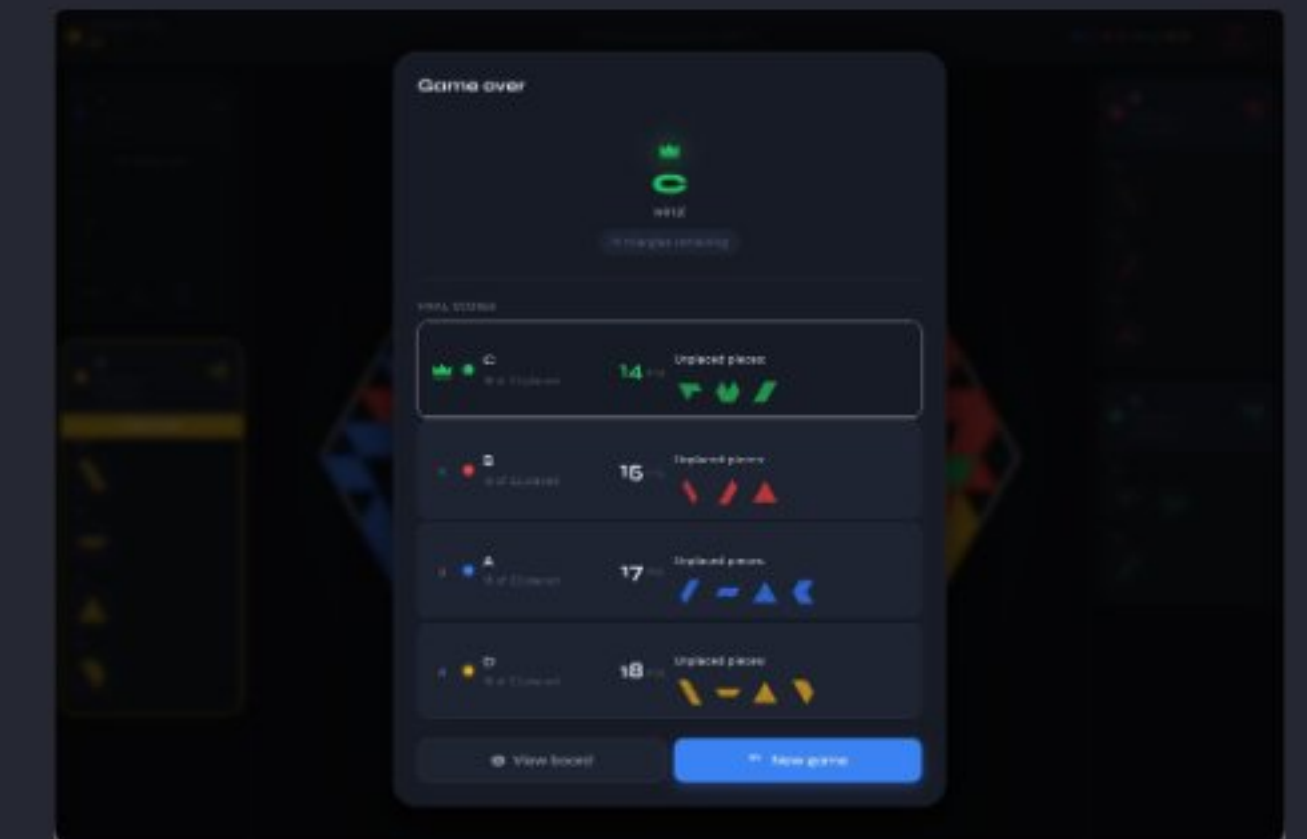
Share link or join by code,
choose game modes

See Movements



Players are able to see each
other's moves on the board

Updated Scoring

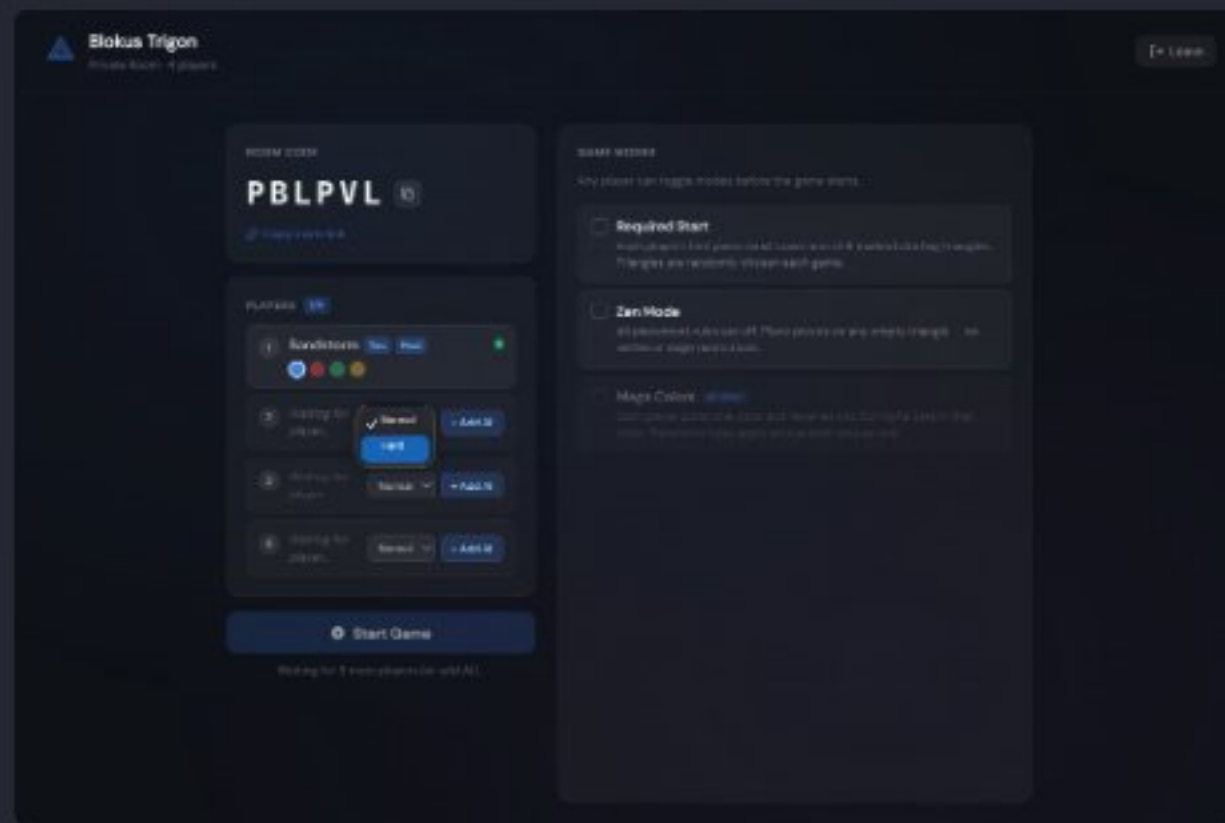


Shows unplaced pieces

Phase 4: AI + Polish

AI opponents, timers, how-to-play, final UI

AI Opponents



Add Normal or Hard difficulty AI opponents

Adjust Scoreboard



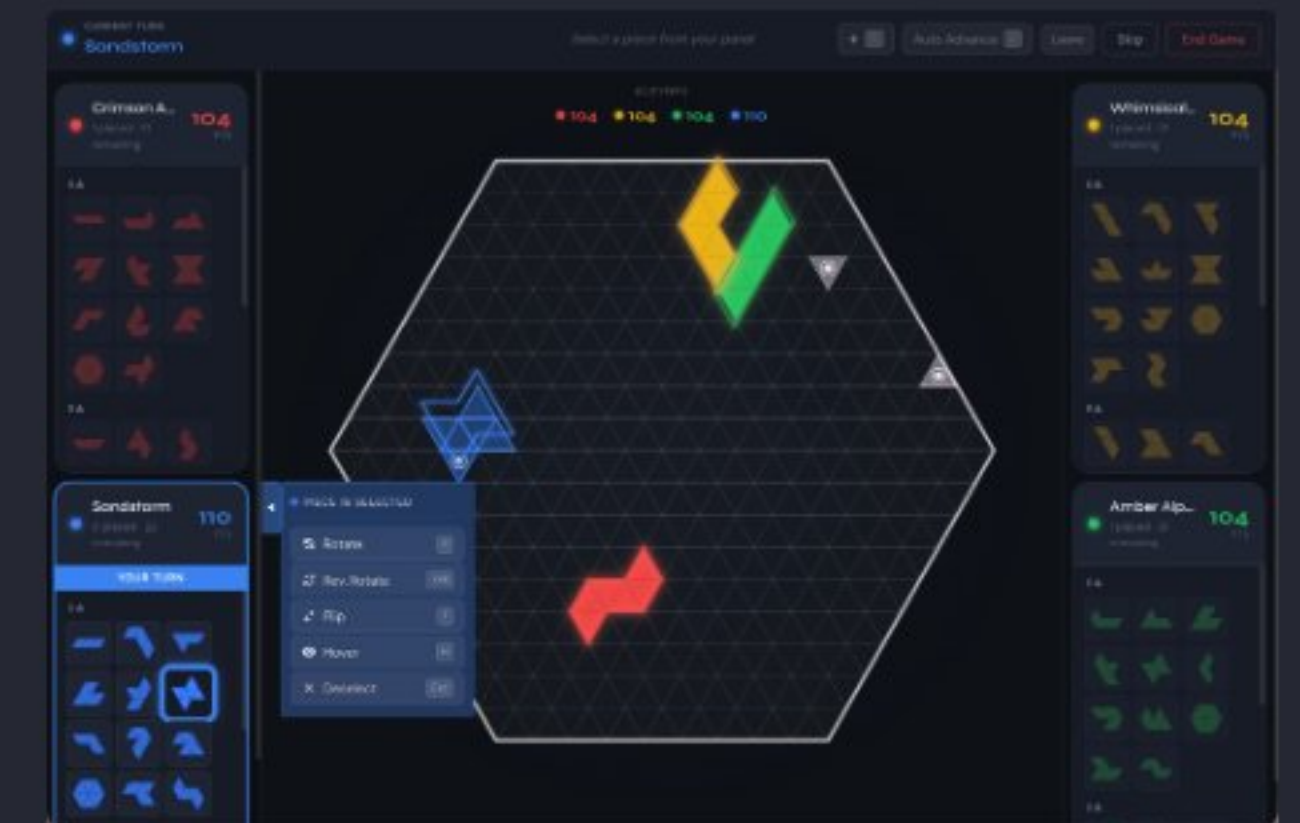
Scoreboard is on top of the board, more visible

Turn Indicator



Board lights up if player hasn't taken their turn

HUD Control Panel

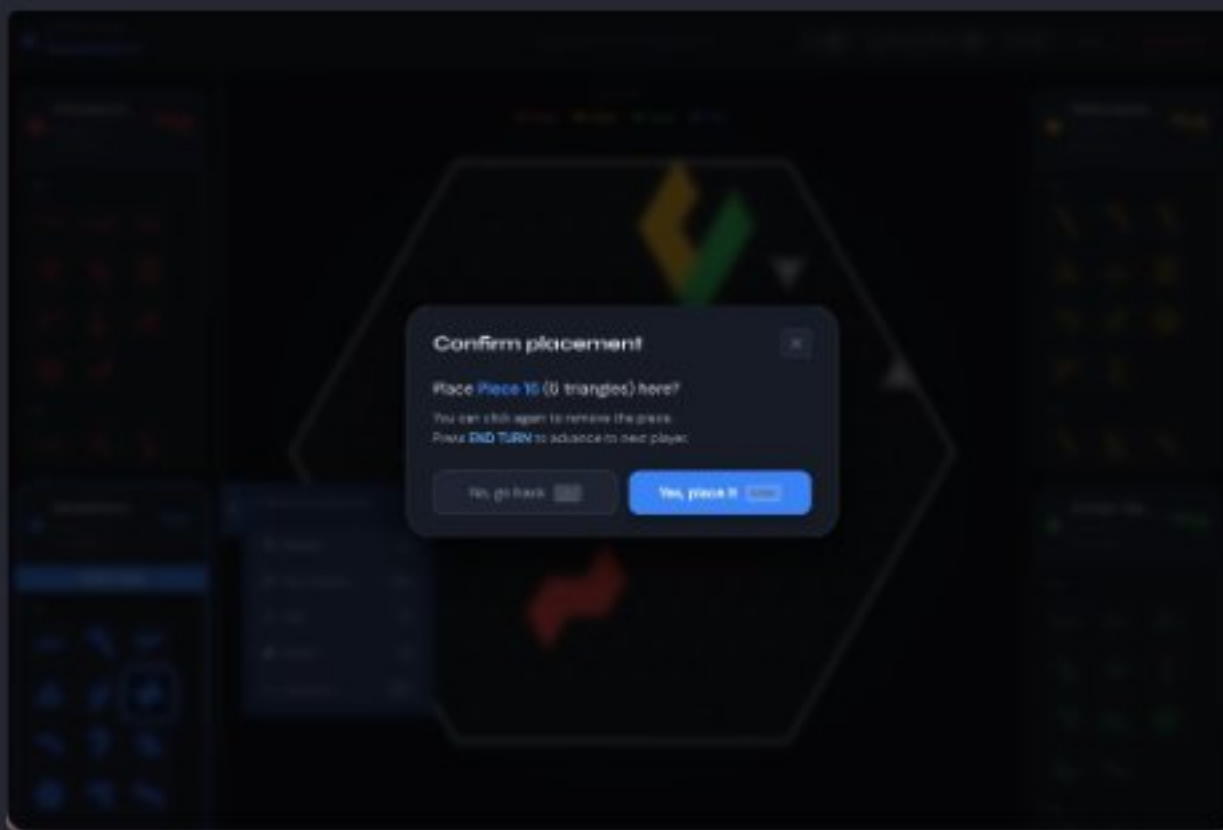


Each player can see their controls, consolidated

Phase 4: AI + Polish

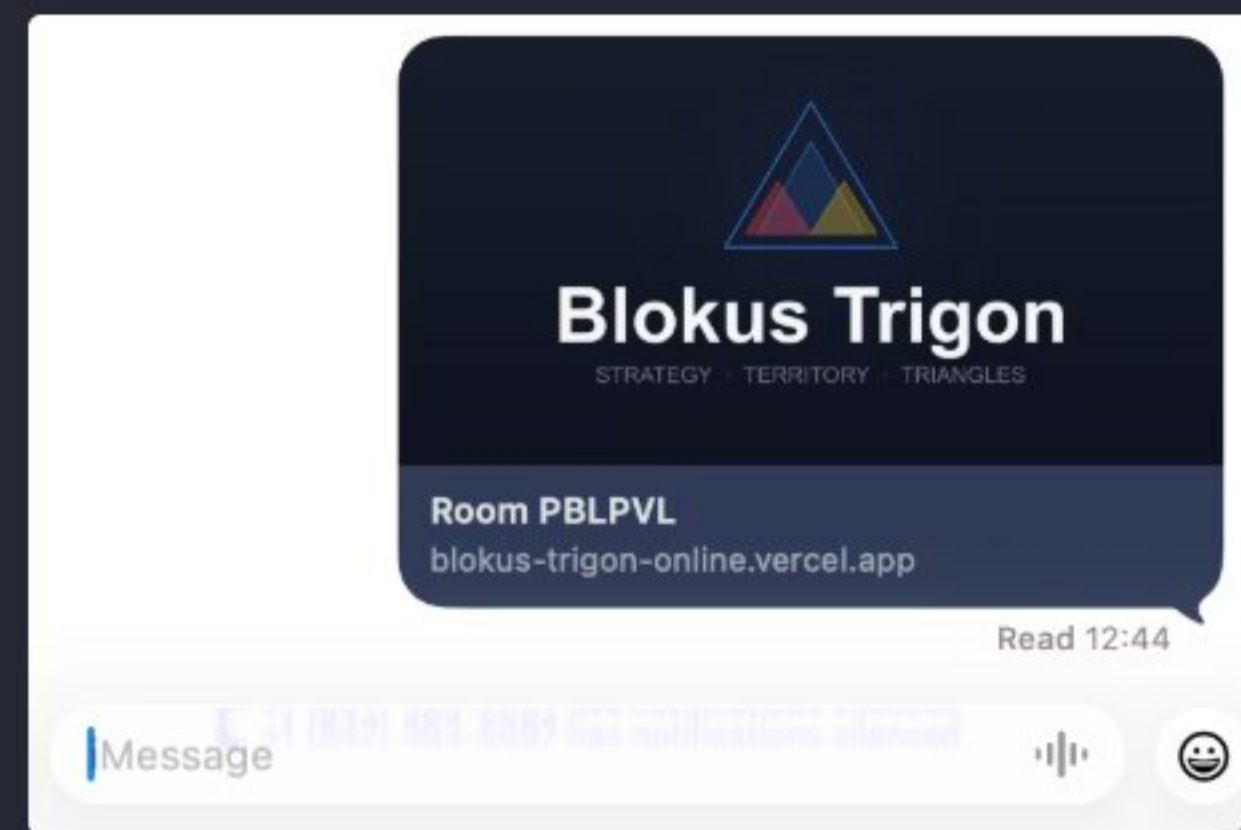
AI opponents, timers, how-to-play, final UI

Turn Advancement



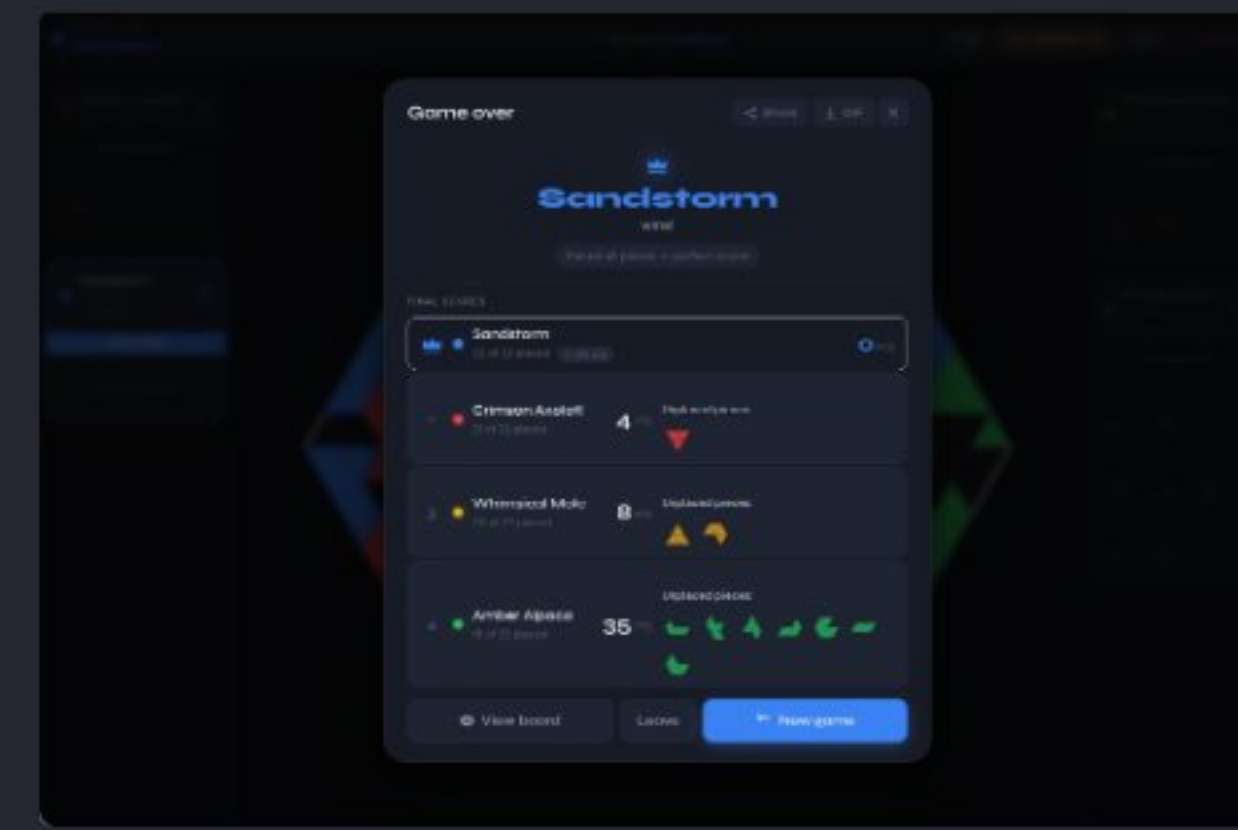
Choice between extra "End Turn" button or "Auto Adv."

Updated Share Link



Add logo and room code to join game message

Updated Scoring



Remaining pieces, timer, and share game

Share & Save Game

